DG Interns Hub – Cybersecurity Internship

WEEK 8 ASSIGNMENT REPORT: ADVANCED VAPT

Author: B Sai Amulya Goud

Intern Id: DG/SEPTEMBER/CYBER/179

Program: Cybersecurity Virtual Internship

Submission Date: October 28, 2026

Program Details:

Start Date: September 1, 2026

End Date: November 30, 2026

Duration: 3 Months

Location: Remote

Stipend: Unpaid

TABLE OF CONTENTS

1. Objectives 3

2. Target Environment and Scope 4

3. Attack Surface Mapping 5

4. Methodology and Workflow 6

5. SQL Injection Endpoint: Request Analysis 7

6. Additional Module Review 9

   6.1 Brute Force Module 9

   6.2 Weak Session IDs 10

   6.3 File Inclusion 10

7. Logic-Based Testing and Attack Chain 11

8. Risk Summary 13

9. Recommended Security Controls 13

10. Conclusion 14

Appendix: Screenshot Checklist 15

Thank You 15

OBJECTIVES

A vulnerability assessment becomes more meaningful when individual weaknesses are considered in combination. In practical attacks, several small defects or unsafe design choices may support a larger compromise, such as unauthorized access, exposure of database information, or access to server-side files.

This Week 8 exercise examined selected modules in a locally hosted DVWA instance configured at medium security. The work emphasized manual request inspection, parameter behaviour, session-token observations, and analysis of how confirmed and unconfirmed findings might relate to one another.

1. **Attack Surface Mapping:** Identify login forms, input fields, APIs, endpoints, and application workflows.
2. **Advanced Vulnerability Testing:** Investigate business logic flaws, parameter tampering, IDOR, and session management weaknesses.
3. **Attack Chain Creation:** Combine verified vulnerabilities into a step-by-step attack scenario to demonstrate potential security impact.
4. **Burp Suite Exploitation:** Use Proxy and Repeater to intercept, modify, and replay HTTP requests and analyse server responses.
5. **Proof of Concept (POC):** Collect screenshots and request-response evidence to demonstrate each confirmed vulnerability and attack-chain step.
6. **Risk and Impact Analysis:** Evaluate potential data exposure, account compromise, privilege escalation, and business impact.
7. **Remediation Strategy:** Recommend security fixes for individual vulnerabilities and the complete attack chain.
8. **Professional Reporting:** Prepare a detailed VAPT report and presentation documenting the methodology, findings, evidence, impact, and recommendations.

Area	Current evidence-based status
Attack surface mapping	Paths visible in Burp Site map; requests captured in HTTP history.
SQL Injection endpoint	POST /vulnerabilities/sqli/; parameters id and Submit; HTTP 200 observed.
Weak Session IDs	Not yet verified in the evidence reviewed for this report.
File Inclusion	Endpoint appears in Site map; exploitability not verified.
End-to-end attack chain	Not demonstrated yet; label as proposed unless later evidence confirms it.

2. Target Environment and Scope

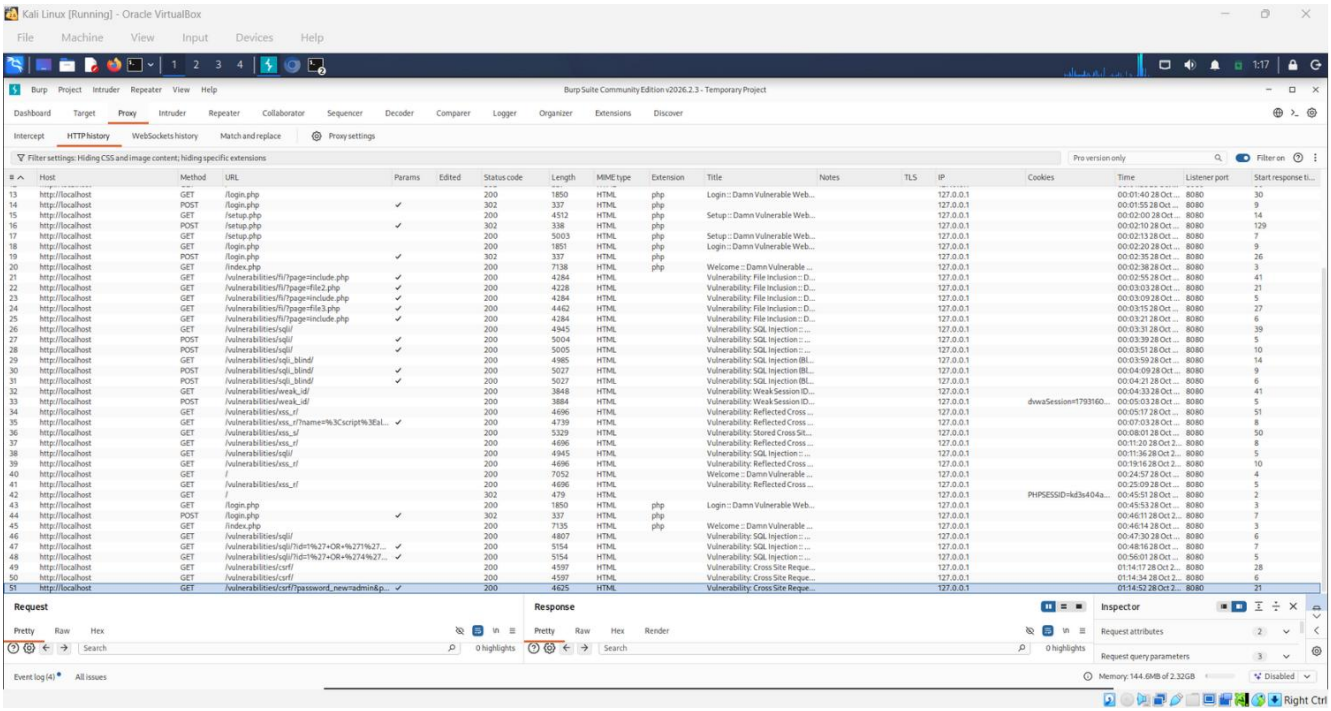
Item	Description
Target	DVWA on localhost
Security setting	Medium
Operating environment	Kali Linux running in Oracle VirtualBox
Proxy/testing tool	Burp Suite Community Edition
Browser	Burp built-in browser
Approach	Manual navigation, HTTP history review, Site map review, request inspection
Scope limitation	Local lab application only; no external targets tested.

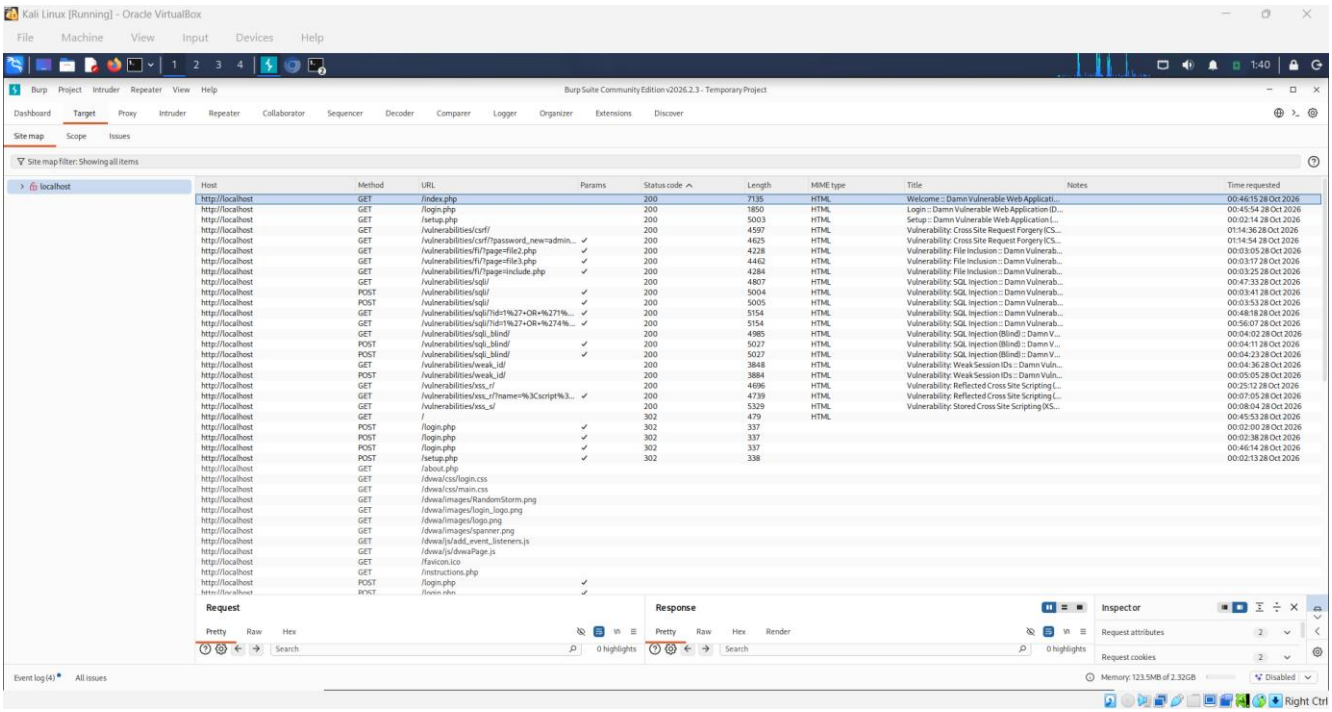
Testing was limited to the local practice environment. Findings should be updated only when supported by your own screenshots or repeatable observations.

3. Attack Surface Mapping

Attack surface mapping records reachable pages and the inputs the application accepts. The DVWA host was expanded in Target → Site map, and the filter was changed to show all items. Visible paths included login and security pages and several vulnerability modules.

Observed endpoint / path	Observed method or input	Purpose / notes
/login.php	GET/POST visible	Login interface.
/security.php	GET/POST visible	DVWA security configuration.
/vulnerabilities/sqli/	POST confirmed	id and Submit parameters captured.
/vulnerabilities/sqli_blind/	GET/POST visible	Separate blind SQLi module; not the same endpoint.
/vulnerabilities/brute/	Path visible	Login-testing module; request details pending.
/vulnerabilities/csrf/	GET visible	CSRF module; testing not documented yet.
/vulnerabilities/fi/	GET with page parameter visible	File Inclusion input surface only.
/vulnerabilities/weak_id/	Path visible	Token behaviour not yet verified.
/vulnerabilities/xss_r/	GET with name parameter visible	Reflected XSS module; Week 8 result not documented here.
/vulnerabilities/xss_s/	Path visible	Stored XSS module; Week 8 result not documented here.


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.2.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Request
Response
Inspector
Right Ctrl


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.2.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Site map Scope Issues
Request
Response
Inspector
Right Ctrl

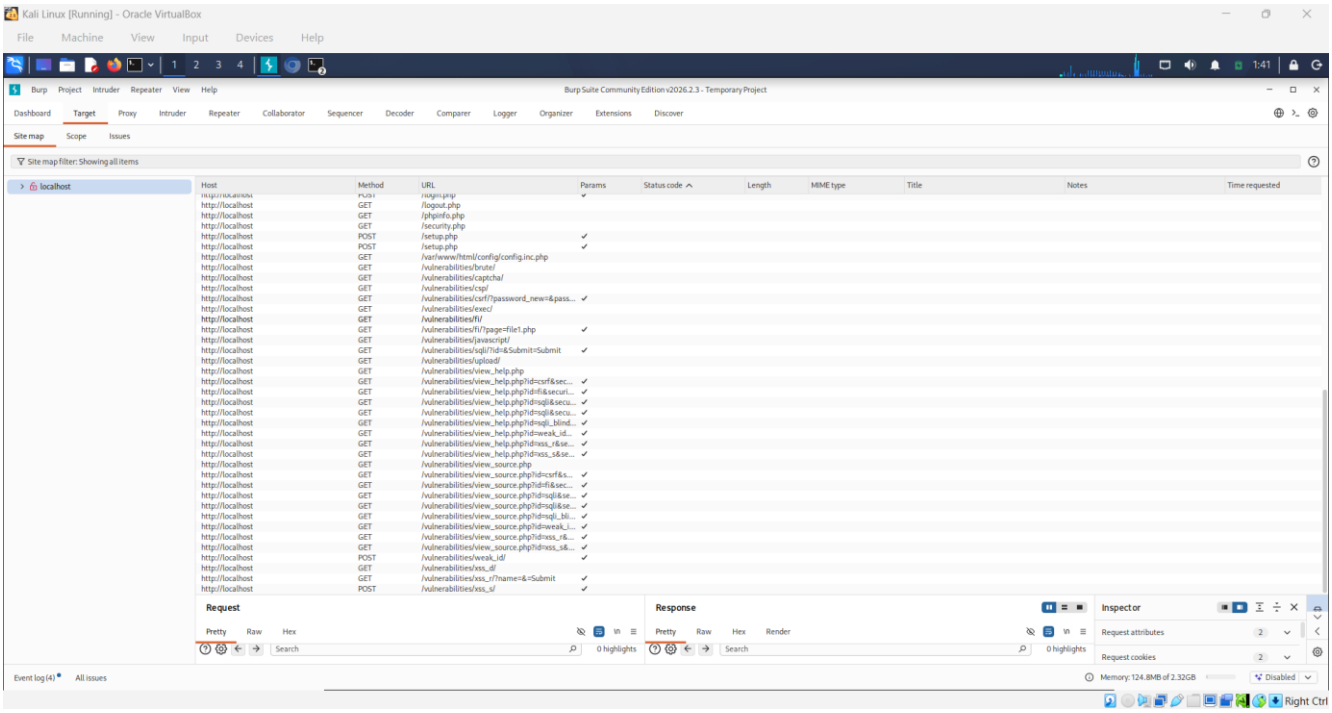


Figure 1. DVWA attack surface displayed in Burp Suite Site map.

4. Methodology and Workflow

1. Confirmed DVWA was accessible through the Burp built-in browser.
2. Confirmed DVWA security was set to Medium.
3. Visited DVWA pages to populate Burp HTTP history and the Site map.
4. Reviewed request paths, methods, and visible parameters.
5. Selected the POST request to /vulnerabilities/sqli/ and reviewed its request body.
6. Reserved vulnerability conclusions for results supported by repeatable evidence.

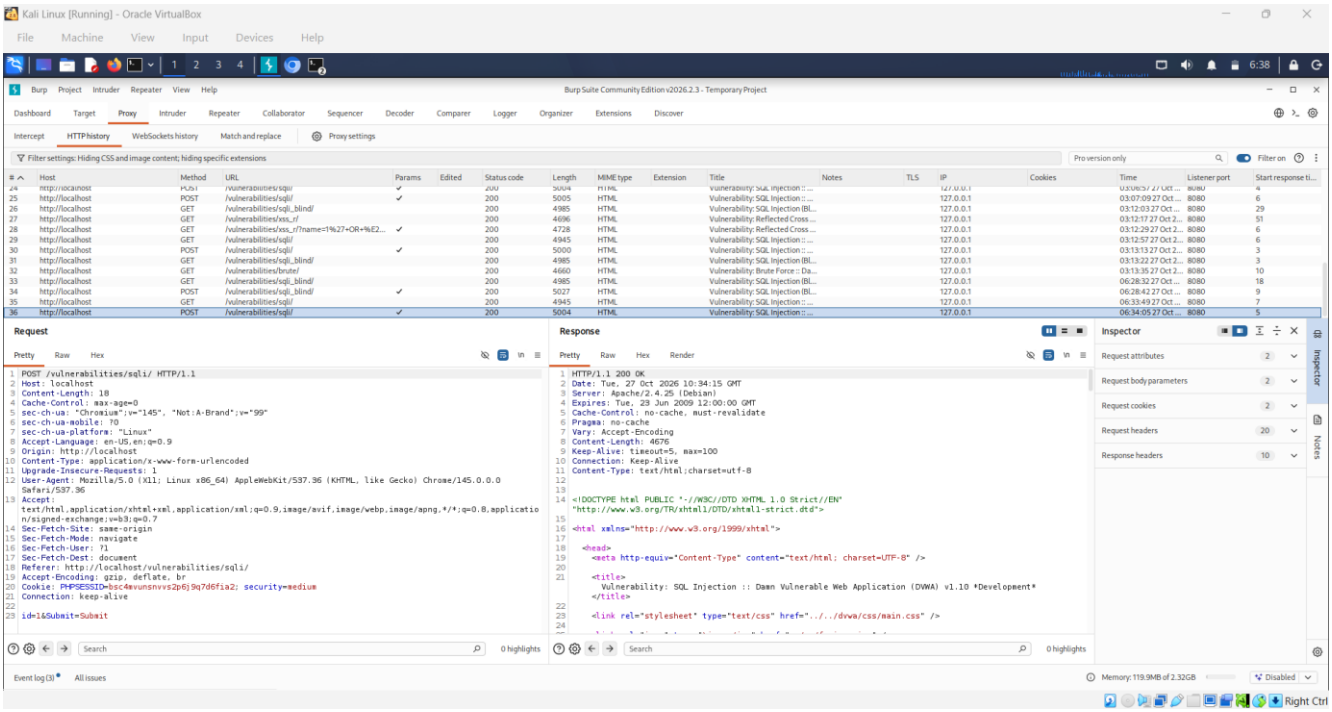


Figure 2. HTTP history used to review requests generated by DVWA.

5. SQL Injection Endpoint: Request Analysis

A request to the regular SQL Injection module was captured in Burp Proxy → HTTP history. The selected entry used POST and the path /vulnerabilities/sqli/. The request body included id=1&Submit=Submit, and the response status was 200 OK.

Property	Observed value
HTTP method	POST
Endpoint	/vulnerabilities/sqli/
Request body parameters	id=1; Submit=Submit
Response status	200 OK
Interpretation	Confirms request processing; does not alone prove SQL injection.

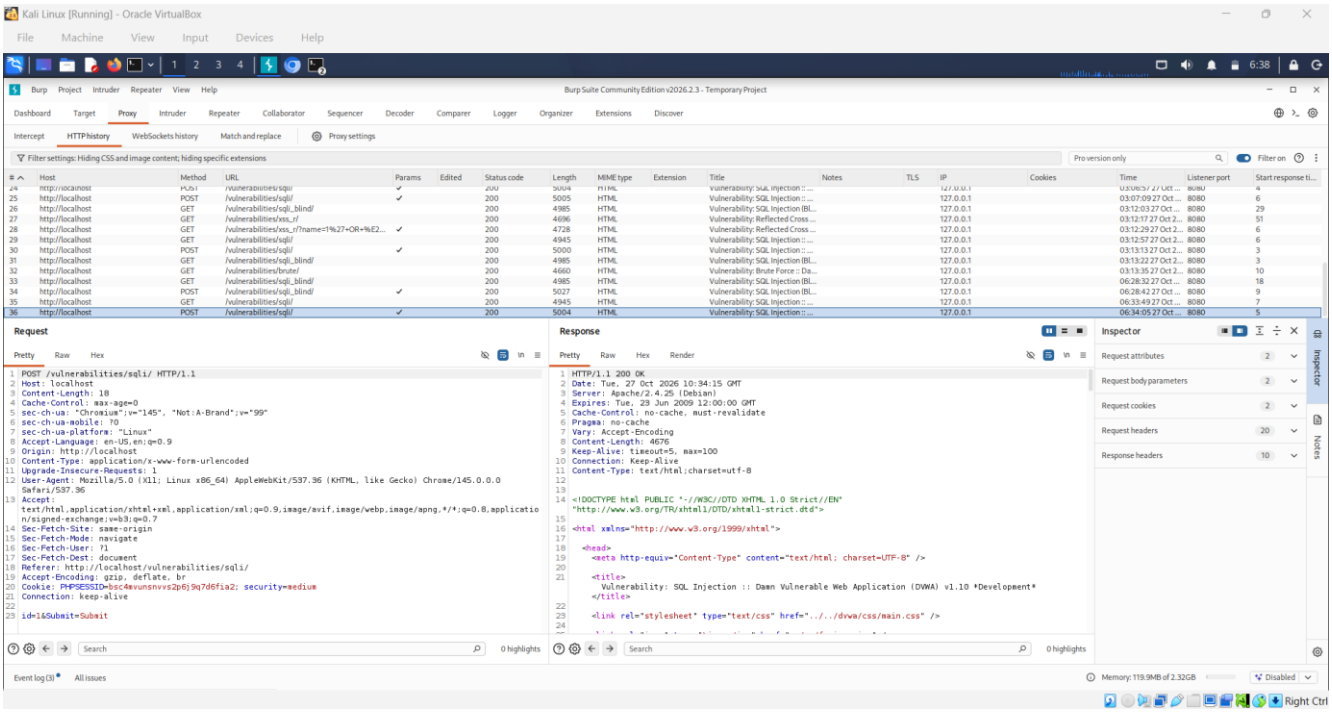


Figure 3. Captured SQL Injection form submission and parameters.

Parameter purpose: id carries the selected user identifier; Submit is the form's submit-button value. For further validation, send the request to Repeater and compare a baseline response with carefully controlled test inputs. Record the exact observed result; do not claim an injection flaw unless evidence supports it.

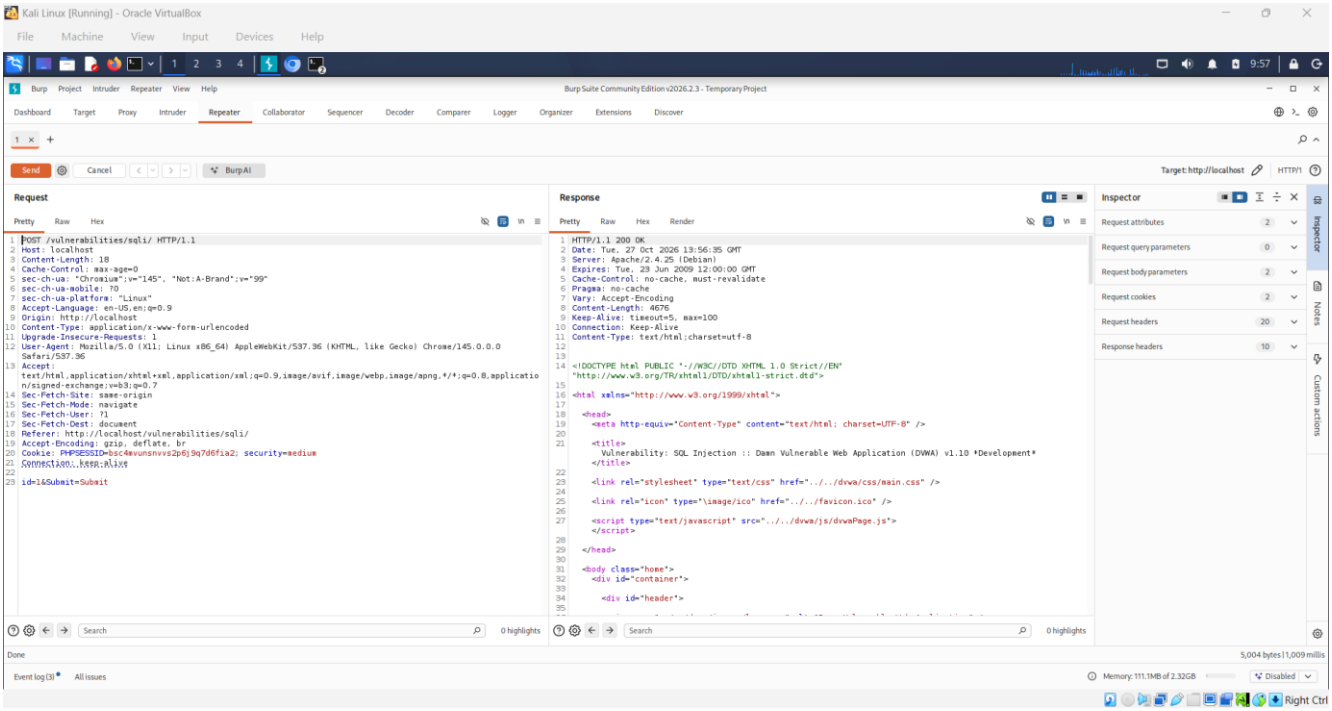


Figure 4. Baseline request/response evidence.

6. Additional Module Review

6.1 Brute Force Module

The Brute Force endpoint appears in the Site map. To complete this section, capture one normal test login request in HTTP history and record the parameter names and response. A single rejected login does not establish whether rate limiting or account lockout is effective.

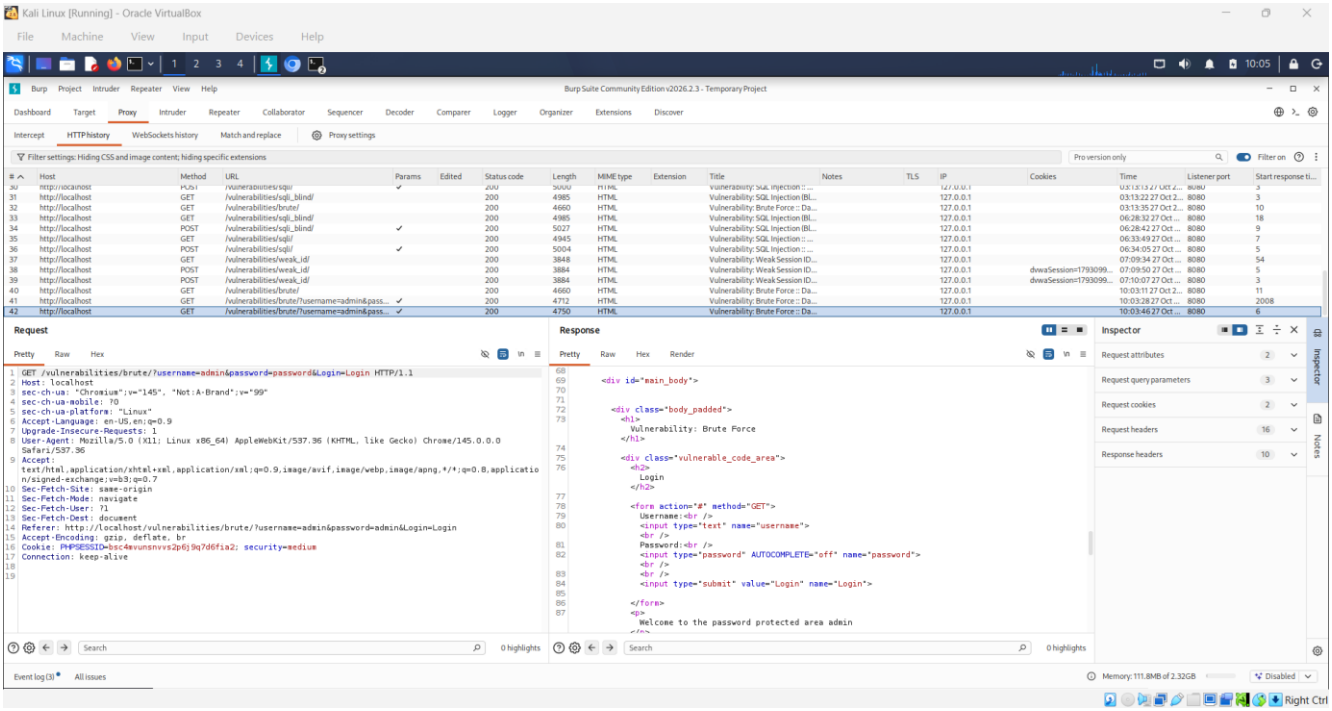


Figure 5. Brute Force request evidence.

6.2 Weak Session IDs

The /vulnerabilities/weak_id/ path is visible in the Site map. This identifies a module but does not confirm predictable tokens. To report a weakness, capture values from your own lab session and explain how they were obtained.

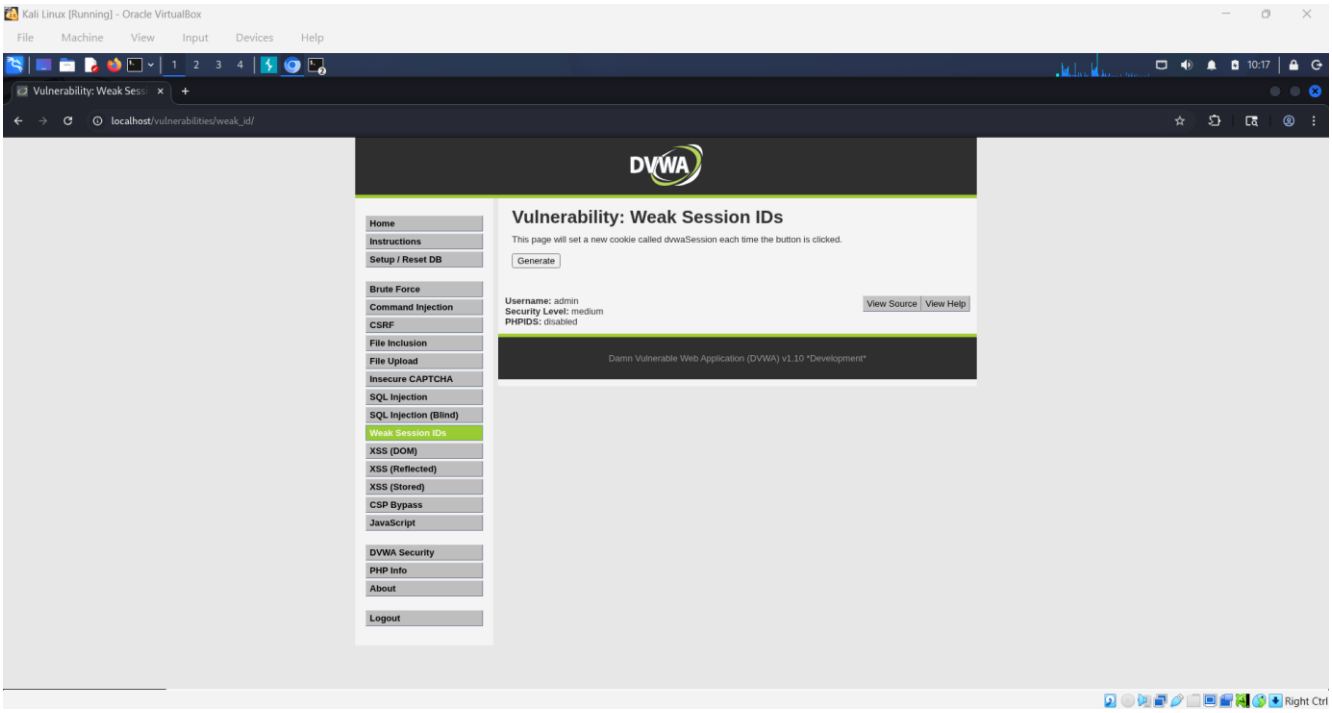


Figure 6. Session ID module evidence.

6.3 File Inclusion

The /vulnerabilities/fi/ endpoint appears in the Site map with a page parameter. This is an identified input surface only. No file disclosure or path traversal result is claimed in this draft.

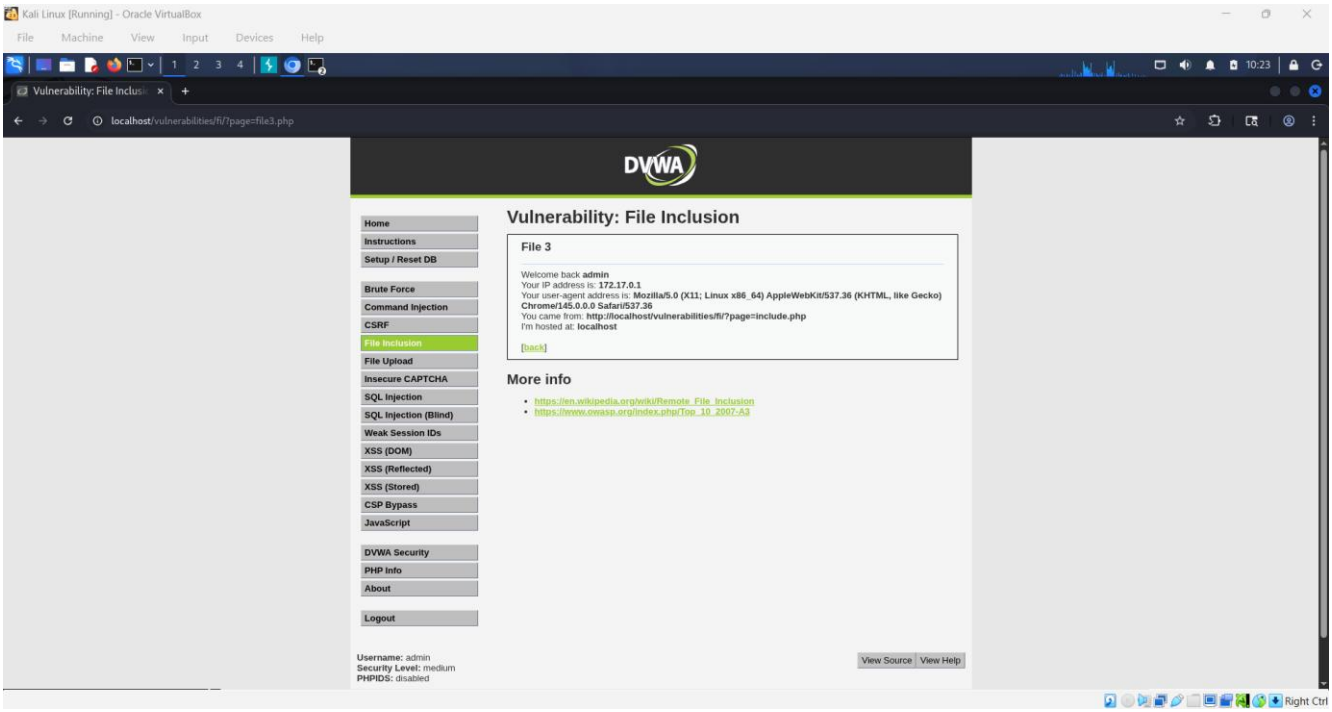


Figure 7. File Inclusion input surface.

7. Logic-Based Testing and Attack Chain

An attack chain describes how verified weaknesses could combine to increase impact. Based on the evidence currently available, an end-to-end chain has not been demonstrated. The current work confirms mapped application paths and a captured SQL Injection form request. Further testing is required before describing a multi-stage compromise as completed.

Stage	Current status	Evidence needed
Identify reachable modules	Observed	Site map screenshot.
Inspect SQLi request structure	Observed	HTTP history showing POST path and parameters.
Verify a security weakness	Observed	Controlled request/response comparison with repeatable result.
Link findings into a chain	Observed	Evidence one verified issue enables the next.
Demonstrate impact	Observed	Safe, local-lab proof of the claimed impact.

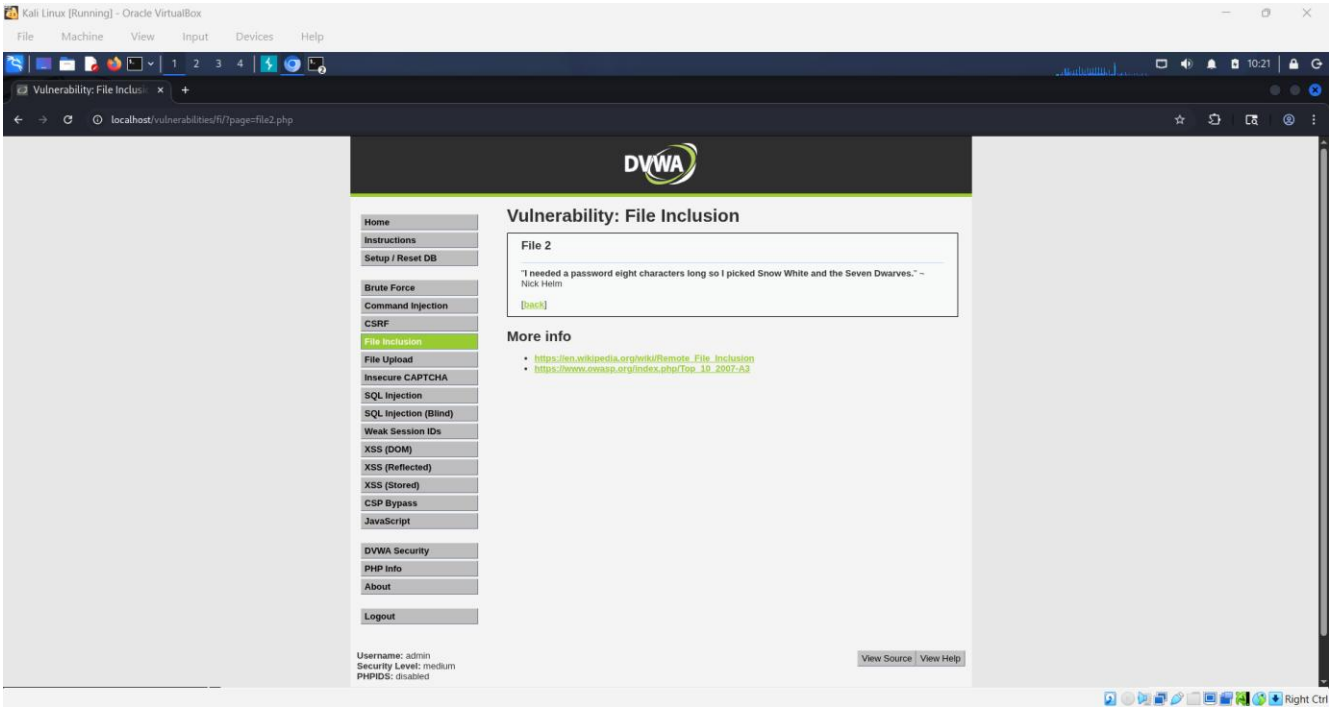
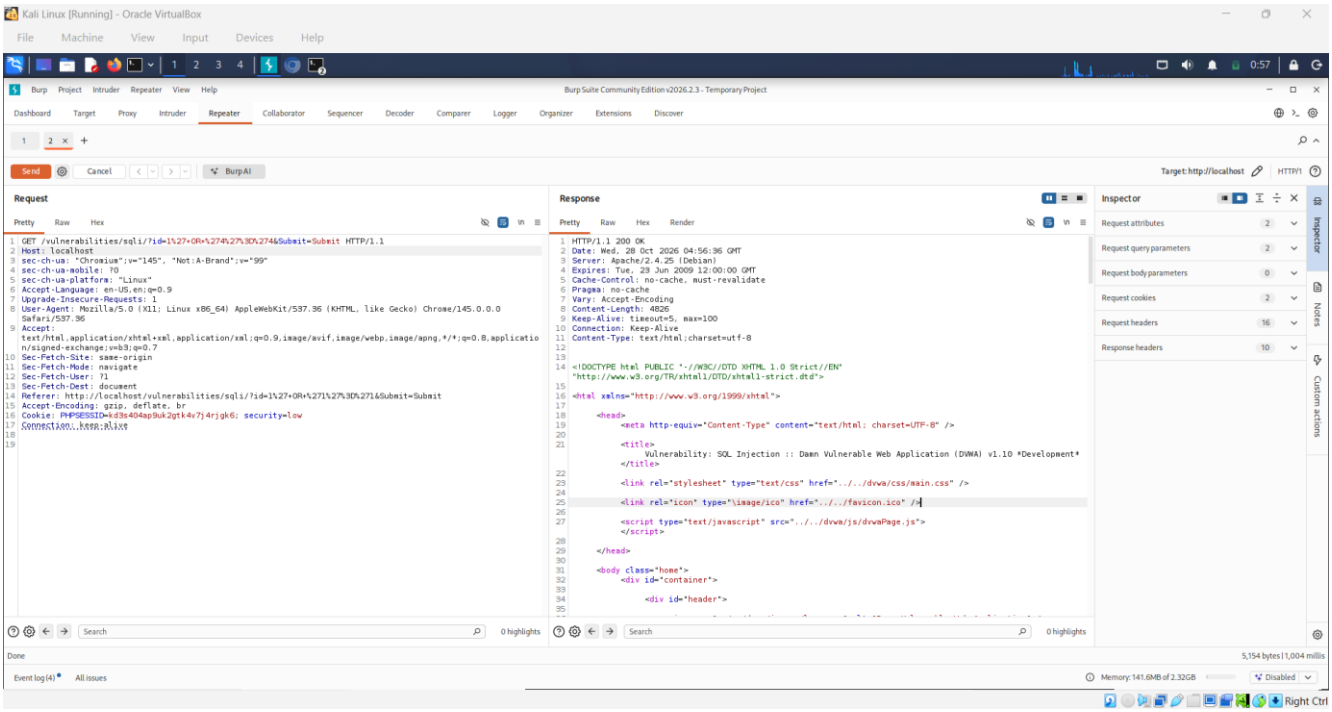
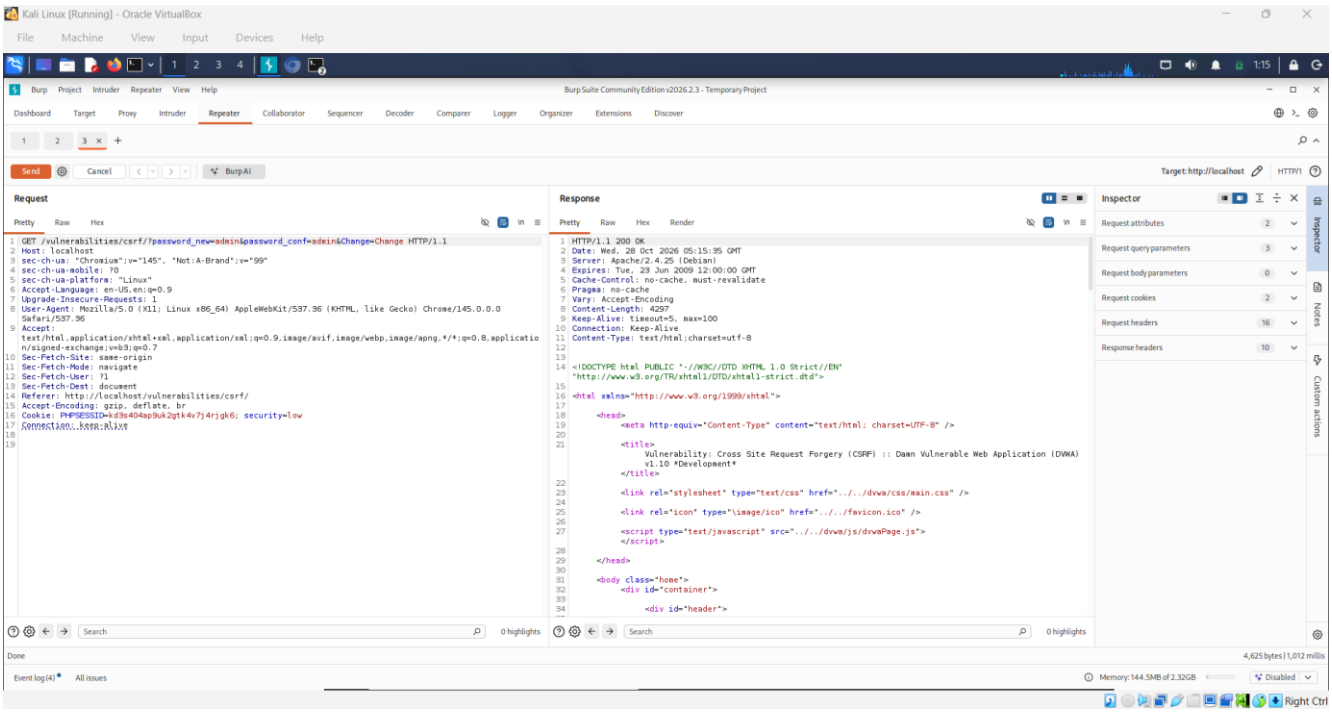


Figure 8. Evidence for a verified chain step.


Kali Linux [Running] - Oracle VirtualBox
Burp Suite Community Edition v2026.2.3 - Temporary Project
Request
GET /vulnerabilities/sqli/?id=1%27+OR+%274%27%3D%274&Submit=Submit HTTP/1.1
Host: localhost
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://localhost/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=kd3s404ap9uk2gtk4v7j4rjgk6; security=low
Connection: keep-alive
Response
HTTP/1.1 200 OK
Date: Wed, 28 Oct 2026 04:56:36 GMT
Server: Apache/2.4.25 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4825
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>
Vulnerability: SQL Injection :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*
</title>
<link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
<link rel="icon" type="\image/ico" href="../../favicon.ico" />
<script type="text/javascript" src="../../dvwa/js/dvwaPage.js">
</script>
</head>
<body class="home">

Inspector
Request attributes
Request query parameters
Request body parameters
Request cookies
Request headers
Response headers
Done
5,154 bytes | 1,004 millis
Memory: 141.6MB of 2.32GB
Right Ctrl


Kali Linux [Running] - Oracle VirtualBox
Burp Suite Community Edition v2026.2.3 - Temporary Project
Request
GET /vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Change HTTP/1.1
Host: localhost
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://localhost/vulnerabilities/csrf/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=kd3s404ap9uk2gtk4v7j4rjgk6; security=low
Connection: keep-alive
Response
HTTP/1.1 200 OK
Date: Wed, 28 Oct 2026 05:15:35 GMT
Server: Apache/2.4.25 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4297
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>
Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*
</title>
<link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
<link rel="icon" type="\image/ico" href="../../favicon.ico" />
<script type="text/javascript" src="../../dvwa/js/dvwaPage.js">
</script>
</head>
<body class="home">

Inspector
Request attributes
Request query parameters
Request body parameters
Request cookies
Request headers
Response headers
Done
4,625 bytes | 1,012 millis
Memory: 144.5MB of 2.32GB
Right Ctrl

RISK SUMMARY

Finding / observation	Status	Potential concern	Recommended control
DVWA endpoints mapped	Confirmed observation	Unnecessary or poorly protected routes expand attack surface.	Remove unused routes; apply authentication and authorization.
SQLi POST request captured	Request confirmed; vulnerability not yet proven by this request alone	If unsafe SQL is verified, database data could be exposed or changed.	Use prepared statements and server-side type validation.
File Inclusion page parameter	Attack surface identified only	Weak path validation may allow unintended file inclusion.	Use a strict server-side allowlist.
Weak Session IDs path	Module identified; token weakness not verified	Predictable tokens may risk unauthorized access if confirmed.	Use cryptographically secure random session identifiers.

Assign severity only after validating the issue and its impact. A module or parameter existing does not, by itself, mean it is vulnerable.

9. Recommended Security Controls

- Use parameterized prepared statements for database queries and validate numeric identifiers server-side.
- Generate session identifiers with a cryptographically secure random source and regenerate them after authentication or privilege changes.
- Use strict server-side allowlists for file inclusion; do not derive arbitrary file paths directly from request parameters.
- Apply authorization checks to sensitive operations and object references.
- Use server-side CSRF token validation for state-changing actions.
- Encode untrusted output according to context to reduce cross-site scripting risk.
- Return generic user-facing errors and store detailed diagnostics in protected logs.
- Keep screenshots, request/response evidence, and conclusions consistent; distinguish confirmed findings from untested possibilities.

Conclusion

This Week 8 assessment began by mapping the local DVWA application using Burp Suite Community Edition. The Site map displayed multiple module paths, and HTTP history confirmed a POST request to the SQL Injection endpoint with id and Submit parameters. These results provide a baseline for further manual testing. Additional checks are required before claiming a confirmed vulnerability or completed attack chain. Finalise this report using your own screenshots and the exact results of tests actually performed.

Appendix: Screenshot Checklist

Photo	Where to capture it	Insert after
1	Target → Site map; localhost expanded; show all items	Section 3
2	Proxy → HTTP history showing DVWA requests	Section 4
3	HTTP history; POST /vulnerabilities/sqli/; request body visible	Section 5
4	Repeater baseline request/response, after performing it	Section 5
5	Brute Force request in HTTP history, after capturing one test login	Section 6.1
6	Weak Session IDs page / your own observation, if tested	Section 6.2
7	File Inclusion page with normal page parameter, if captured	Section 6.3
8	Evidence for any later verified attack-chain step	Section 7

THANK YOU